Apri indice del corso

Stato	Completato
Iniziato	venerdì, 12 giugno 2026, 09:49
Terminato	venerdì, 12 giugno 2026, 10:01
Tempo impiegato	12 min. 11 secondi

Domanda 1

Risposta errata

Lavorando su un progetto .NET grande/complesso, perché è importante utilizzare un file di lock (`packages.lock.json`) insieme al file di progetto (`.csproj`)?

a. Per non inserire troppe informazioni nel file `.csproj` che, in caso di molte dipendenze, diventerebbe troppo grande

b. Il file di lock "congela" l'intero albero delle dipendenze, garantendo che la build sia riproducibile ✓

c. Facendo "lock" il codice generato è più efficiente

d. Senza il lock file, una build potrebbe funzionare oggi ma "rompersi" domani ✓

e. Il file `.csproj` definisce solo le dipendenze dirette, ma non fissa le versioni esatte delle dipendenze transitive ✓

Le risposte corrette sono: Il file `.csproj` definisce solo le dipendenze dirette, ma non fissa le versioni esatte delle dipendenze transitive, Il file di lock "congela" l'intero albero delle dipendenze, garantendo che la build sia riproducibile, Senza il lock file, una build potrebbe funzionare oggi ma "rompersi" domani

Domanda 2

Risposta corretta

Quali proprietà valgono per i value types in C#?

a. Includono `struct`, `enum`, tipi numerici e `bool` ✓

b. Sono sempre allocati sullo heap come oggetti distinti

c. Sono `null` di default

d. L'assegnazione copia il valore ✓

Le risposte corrette sono: L'assegnazione copia il valore, Includono `struct`, `enum`, tipi numerici e `bool`

Domanda 3

Risposta corretta

Quali operazioni sono tipicamente possibili via reflection?

a. Generare automaticamente unit-test
b. Creare nuove istanze con `new`
c. Creare istanze con `Activator.CreateInstance(...)` ✓
d. Invocare metodi con `MethodInfo.Invoke(...)` ✓
e. Caricare assembly e scoprire tipi a runtime (plugin) ✓

Le risposte corrette sono: Creare istanze con `Activator.CreateInstance(...)`, Invocare metodi con `MethodInfo.Invoke(...)`, Caricare assembly e scoprire tipi a runtime (plugin)

Domanda 4

Risposta corretta

Quali affermazioni descrivono correttamente Git come DVCS (Distributed VCS)?

a. Operazioni come `commit` e consultazione `log` sono tipicamente molto veloci perché avvengono in locale ✓
b. Ogni clone contiene l'intera cronologia del repository ✓
c. Senza connessione Internet non si può fare `commit`
d. Esiste un unico server centrale, GitHub, senza il quale il sistema non funziona

Le risposte corrette sono: Ogni clone contiene l'intera cronologia del repository, Operazioni come `commit` e consultazione `log` sono tipicamente molto veloci perché avvengono in locale

Domanda 5

Parzialmente corretta

`run:` vs `uses:` in un step di GitHub Actions: quali affermazioni sono corrette?

a. `uses:` richiama una action (es. `actions/checkout@v4`) ✓
b. `run:` esegue comandi di shell ✓
c. `uses:` serve solo per stampare stringhe
d. `run:` può essere usato per eseguire `dotnet test` ✓
e. sono degli alias, si possono usare in modo intercambiabile

Le risposte corrette sono: `run:` esegue comandi di shell, `uses:` richiama una action (es. `actions/checkout@v4`), `run:` può essere usato per eseguire `dotnet test`

Domanda 6

Risposta corretta

Quali problemi rientrano nel classico "it works on my machine"?

a. Variabili d'ambiente/percorsi/credenziali non documentate ✓
b. Gli sviluppatori non hanno abbastanza spazio su disco
c. Dipendenze implicite installate solo sulla macchina dello sviluppatore ✓
d. Il garbage collector non funziona in produzione
e. Il continuo spostamento della data d'uscita di GTA 6

Le risposte corrette sono: Dipendenze implicite installate solo sulla macchina dello sviluppatore, Variabili d'ambiente/percorsi/credenziali non documentate

Domanda 7

Risposta corretta

Quali affermazioni sono vere riguardo la *staging area (index)* in Git?

a. Consente di costruire commit "pezzo per pezzo" ✓
b. Permette di selezionare quali modifiche includere nel prossimo commit ✓
c. È un backup automatico su GitHub
d. Funziona solo in modalità online
e. È obbligatoria solo nei repository molto grandi

Le risposte corrette sono: Permette di selezionare quali modifiche includere nel prossimo commit, Consente di costruire commit "pezzo per pezzo"

Domanda 8

Parzialmente corretta

Stai scrivendo un Dockerfile. Devi inserire i file del codice sorgente nell'immagine. Perché la documentazione e le best practice suggeriscono di usare `COPY . .` invece di `ADD . .` se non hai esigenze particolari?

a. Perché `ADD` funziona solo in modalità rootless
b. Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie ✓
c. Non c'è differenza, sono alias dello stesso comando
d. Perché `COPY` crea un layer più piccolo rispetto ad `ADD`

La risposta corretta è: Perché `ADD` ha funzionalità aggiuntive (come scaricare file da URL remoti o decomprimere archivi locali) che rendono il comportamento meno prevedibile se non strettamente necessarie

Fine revisione